프로그래밍 언어 원리와 코드 품질

Programming Language Principles and Code Quality

AI 시대의 프로그래밍 언어론

학습 내용

- 1. AI 시대, 개발자에게 필요한 능력
- 2. 프로그래밍 언어의 역사와 발전
- 3. 좋은 코드란 무엇인가?

AI 시대의 현실

AI가 할 수 있는 것

- ✓ 빠른 코드 생성
- ✓ 반복적인 패턴 작성
- ✓ 기본적인 알고리즘 구현
- ✓ 문서화 및 주석 생성

개발자가 해야 할 일

- ★ 코드의 정확성 검증
- ★ 성능 및 효율성 평가
- ★ 보안 취약점 식별
- ★ 유지보수 가능성 판단
- ★ 최적의 해결책 선택

AI는 도구임

좋은 코드를 식별하고 선택할 수 있는 능력

→ 탄탄한 기본기가 필수!

주요 테크 기업의 원칙

> "
>
> ***AI 코드도 결국 개발자가 책임지고 통제할 수 있어야 한다***
>
> - 주요 테크 기업 공통 원칙

✓ 모든 AI 생성 코드는 엔지니어의 검토와 승인 필수

프로그래밍 언어의 역사

1950s	Assembly	기계어에 가까운 저수준 언어
1970s	C	시스템 프로그래밍의 표준
1980s	C++	객체지향 프로그래밍 도입
1990s	Java, Python	생산성과 이식성 강화
2000s~	현대 언어들	안전성, 동시성, 표현력 향상

왜 여러 언어가 존재하는가?

문제 영역

시스템 프로그래밍
웹 개발
데이터 분석
→ 각기 다른 요구사항

설계 철학

성능 우선
생산성 우선
안전성 우선
→ 트레이드오프

발전 과정

기술의 진화
하드웨어 변화
개발 패러다임 변화
→ 지속적 혁신

좋은 코드의 기준

1. **정확성 (Correctness)** — 요구사항을 정확히 구현
2. **가독성 (Readability)** — 읽고 이해하기 쉬운 코드
3. **유지보수성 (Maintainability)** — 수정과 확장이 용이
4. **효율성 (Efficiency)** — 시간과 공간 복잡도 최적화

좋은 코드 vs 나쁜 코드

❌ 나쁜 코드

- 변수명: a, b, c, temp
- 주석 없음
- 중복 코드 많음
- 복잡한 중첩 구조
- 매직 넘버 사용

✓ 좋은 코드

- 의미 있는 변수명
- 적절한 주석과 문서화
- 함수 분리와 재사용
- 명확한 구조
- 상수 정의 사용

이 과목에서 배울 것

- 언어의 원리와 작동 방식
  → 왜 이 코드가 작동하는가? 왜 작동하지 않는가?
- 코드 품질을 평가하는 기준
  → AI가 생성한 코드의 문제점을 찾아내는 능력
- 더 나은 코드를 작성하는 방법
  → 안전하고, 효율적이며, 유지보수 가능한 코드

15주 후

- AI 생성 코드의 문제점을 즉시 발견
- 더 나은 대안을 제시할 수 있는 지식
- 언어의 본질을 이해하여 새로운 언어도 빠르게 학습
- 코드의 성능과 안전성 평가

AI 시대의 개발자

AI를 잘 쓰는 것도 중요하지만,
AI가 만든 코드를 제대로 판단하고
책임질 수 있는 능력이 더 중요